

Building a Calculator with Tkinter

Study Notes — GUI Layout, Event Handling & Functions

1. What is Tkinter?

Tkinter is Python's standard built-in library for creating desktop GUI applications. It comes pre-installed with Python, so no extra installation is usually needed. A calculator app is a classic beginner project because it combines widgets (buttons, entry fields), layout management, and functions that respond to button clicks.

Core building blocks we'll use:

- **Tk()** — creates the main application window
- **Entry** — a text box to display numbers/results
- **Button** — clickable widgets for digits and operators
- **Frame / grid()** — arranges widgets in rows and columns
- **Functions** — Python code that runs when a button is clicked (event handlers)

2. Setting Up the Main Window

Every Tkinter app starts by creating a root window, giving it a title, and calling **mainloop()** at the very end to keep it open and listening for events.

```
import tkinter as tk

root = tk.Tk()
root.title("Simple Calculator")
root.geometry("320x420")
root.resizable(False, False)

# ... widgets go here ...

root.mainloop() # keeps the window open, must be the LAST line
```

3. The Display (Entry Widget)

The display shows what the user types and the final result. A **StringVar** links the Entry widget to a Python variable so we can read/update its text easily from functions.

```
display_var = tk.StringVar()

display = tk.Entry(
    root, textvariable=display_var, font=("Arial", 20),
    justify="right", bd=8, relief="ridge"
)
display.grid(row=0, column=0, columnspan=4, padx=10, pady=10, ipady=10)
```

4. Core Functions (Event Handlers)

Each button calls a small function when clicked. Keeping logic in separate functions makes the code reusable and easy to debug.

4.1 click() — appends digits / operators to the display

```
def click(value):
    """Append the pressed value to the current expression."""
    current = display_var.get()
    display_var.set(current + str(value))
```

4.2 calculate() — evaluates the expression

```
def calculate():
    """Evaluate the expression currently shown on the display."""
    try:
        expression = display_var.get()
        result = eval(expression)          # evaluates "12+7*2" etc.
        display_var.set(result)
    except ZeroDivisionError:
        display_var.set("Error: Div by 0")
    except Exception:
        display_var.set("Error")
```

Note: **eval()** is convenient for a learning project, but it can execute arbitrary code if untrusted input is passed to it. For production apps, write a proper expression parser or restrict allowed characters instead of using `eval()`.

4.3 clear() — resets the display

```
def clear():
    """Clear the display completely."""
    display_var.set("")
```

4.4 backspace() — removes the last character

```
def backspace():
    """Delete the last character from the display."""
    current = display_var.get()
    display_var.set(current[:-1])
```

5. Creating Buttons with a Loop

Instead of writing one `Button()` call per key, define a layout list and loop through it. This keeps the code short and makes the calculator easy to extend.

```
buttons = [
    ("7", 1, 0), ("8", 1, 1), ("9", 1, 2), ("/", 1, 3),
    ("4", 2, 0), ("5", 2, 1), ("6", 2, 2), ("*", 2, 3),
    ("1", 3, 0), ("2", 3, 1), ("3", 3, 2), ("-", 3, 3),
    ("0", 4, 0), (".", 4, 1), ("C", 4, 2), ("+", 4, 3),
]

for (text, row, col) in buttons:
    if text == "C":
        btn = tk.Button(root, text=text, width=5, height=2,
                        command=clear, bg="#ffb4a2")
    else:
        btn = tk.Button(root, text=text, width=5, height=2,
                        command=lambda t=text: click(t))
    btn.grid(row=row, column=col, padx=3, pady=3)

# Equals button spans full width on its own row
equals_btn = tk.Button(root, text="=", width=24, height=2,
                      bg="#8ecae6", command=calculate)
equals_btn.grid(row=5, column=0, columnspan=4, padx=3, pady=6)
```

Important: use **`lambda t=text: click(t)`** (a default argument) instead of **`lambda: click(text)`**. Because Python closures capture variables by reference, without the default argument every button would end up calling `click()` with the value of `text` after the loop finishes — usually the last button created.

6. Full Working Example

```
import tkinter as tk

def click(value):
    display_var.set(display_var.get() + str(value))

def calculate():
    try:
        display_var.set(eval(display_var.get()))
    except ZeroDivisionError:
        display_var.set("Error: Div by 0")
    except Exception:
        display_var.set("Error")

def clear():
    display_var.set("")

def backspace():
    display_var.set(display_var.get()[:-1])

root = tk.Tk()
root.title("Simple Calculator")
root.geometry("320x420")
```

```

root.resizable(False, False)

display_var = tk.StringVar()
display = tk.Entry(root, textvariable=display_var, font=("Arial", 20),
                    justify="right", bd=8, relief="ridge")
display.grid(row=0, column=0, columnspan=4, padx=10, pady=10, ipady=10)

buttons = [
    ("7", 1, 0), ("8", 1, 1), ("9", 1, 2), ("/", 1, 3),
    ("4", 2, 0), ("5", 2, 1), ("6", 2, 2), ("*", 2, 3),
    ("1", 3, 0), ("2", 3, 1), ("3", 3, 2), ("-", 3, 3),
    ("0", 4, 0), (".", 4, 1), ("C", 4, 2), ("+", 4, 3),
]

for (text, row, col) in buttons:
    cmd = clear if text == "C" else (lambda t=text: click(t))
    tk.Button(root, text=text, width=5, height=2, command=cmd).grid(
        row=row, column=col, padx=3, pady=3)

tk.Button(root, text="=", width=24, height=2, command=calculate).grid(
    row=5, column=0, columnspan=4, padx=3, pady=6)
tk.Button(root, text="<-", width=24, height=1, command=backspace).grid(
    row=6, column=0, columnspan=4, padx=3, pady=3)

root.mainloop()

```

7. Bonus: Keyboard Input

You can let users type instead of clicking, using `bind()` to connect key presses to functions.

```

root.bind("<Return>", lambda event: calculate())
root.bind("<BackSpace>", lambda event: backspace())
root.bind("<Key>", lambda event: click(event.char) if event.char in "0123456789+-*/." else None)

```

8. Function Summary

Function	Purpose	Triggered by
<code>click(value)</code>	Adds a digit/operator to display	Number & operator buttons
<code>calculate()</code>	Evaluates the full expression	"=" button / Enter key
<code>clear()</code>	Empties the display	"C" button
<code>backspace()</code>	Removes last character	"<-" button / Backspace key

9. Tips & Common Pitfalls

- Always call **`root.mainloop()`** as the last line — the window won't appear otherwise.
- Use **`grid()`** consistently; mixing `grid()` and `pack()` in the same container causes errors.
- Wrap `eval()` in `try/except` so a bad expression (like "5+") doesn't crash the app.

- Use lambda with a default argument (t=text) inside loops to avoid the 'last value' bug.
- For a safer alternative to eval(), consider the built-in **ast.literal_eval** combined with a manual operator parser, or a small expression-evaluation library.